

NYC Taxi Pipeline — Full Project Report

A complete record of what was built, in what order, and every real issue hit along the way.

1. Project goal

Build a complete, portfolio-ready data engineering pipeline: ingest a real public dataset, transform it with dbt, load it into a cloud warehouse, and visualize the results — the kind of end-to-end project that demonstrates practical skills beyond just certifications.

Dataset chosen: NYC Yellow Taxi Trip Data (May 2026), from NYC TLC's public open data.

Final stack: Python → Snowflake → dbt → Looker Studio → GitHub.

2. Phase 1 — Setup

- Created a free Snowflake trial account
- Created the GitHub repo folder locally (taxi/)
- Set up .env for Snowflake credentials, added to .gitignore immediately
- Set up .gitignore and initialized git (git init)
- Installed Python libraries: pandas, pyarrow, snowflake-connector-python, python-dotenv

Issues hit: - git check-ignore failed with “not a git repository” — fixed by running git init first, since the folder hadn't been initialized yet. - Xcode Command Line Tools needed installing before git would work at all on a fresh Mac.

3. Phase 2 — Extract & Load

- Downloaded May 2026 Yellow Taxi trip data as a .parquet file from the NYC TLC site
- Created three Snowflake schemas: RAW, STAGING, MARTS, under a TAXI_PROJECT database
- Created the raw landing table RAW.RAW_TAXI_TRIPS with an explicit schema matching the taxi data columns
- Wrote ingest.py: reads the parquet file with pandas, does light cleaning (drops nulls in key fields, drops negative fares/distances/passenger counts), renames columns to match the Snowflake table, and batch-inserts 10,000 rows at a time via snowflake-connector-python
- Final result: **3,072,944 rows** loaded and verified with SELECT COUNT(*) matching the script's own reported count

Issues hit and fixed: - SSL certificate error (CERTIFICATE_VERIFY_FAILED) when installing Python packages — macOS Python from python.org hadn't set up root certificates. Fixed by running Install Certificates.command from the Python Applications

folder. - **Snowflake timestamp binding error** (Binding data in type (timestamp) is not supported) — the connector's executemany couldn't bind pandas Timestamp objects directly. Fixed by converting pickup_datetime/dropoff_datetime columns to strings (.astype(str)) before inserting; Snowflake auto-parsed them back into proper TIMESTAMP values on the table side.

4. Phase 3 — Transform with dbt

- Installed dbt-snowflake (installs dbt Core + the Snowflake adapter together)
- Ran `dbt init`, connected to Snowflake via `profiles.yml` (account, user, password, warehouse, role, database, schema)
- Built the model layer:
 - `stg_taxi_trips` — staging model reading from `RAW.RAW_TAXI_TRIPS`, casting types, later filtered to exclude bad dates
 - `stg_taxi_zones` — staging model for the taxi zone lookup table (loaded separately via `load_zones.py`)
 - `fct_trips` — fact table joining staged trips with duration and cost-per-mile calculations
 - `agg_daily_summary` — daily rollup: trip count, avg fare, avg distance, avg duration
 - `agg_fare_by_borough` — trips joined to zones, averaged fare by NYC borough
 - `agg_peak_hours` — trip counts by day-of-week and hour-of-day
- Added `schema.yml` files with dbt tests (see Data Quality section below)
- Ran `dbt run` and `dbt test` repeatedly throughout to validate

Issues hit and fixed: - **SSL cert error again** during `pip install dbt-snowflake` (specifically `dbt-core-experimental-parser` trying to download from GitHub) — same root cause and same fix as above (Install Certificates.command + `pip install --upgrade certifi`). - **Wrong password in profiles.yml** caused dbt debug to fail the connection test — fixed by manually editing `~/.dbt/profiles.yml` with the correct Snowflake password. - **Bad date in source data:** a chart later revealed a trip with `pickup_datetime` of 2008-12-31 mixed into the May 2026 data, wrecking the date axis on the volume chart. Fixed by adding a `where pickup_datetime >= '2020-01-01' and pickup_datetime <= current_timestamp()` filter to `stg_taxi_trips`, which cascaded the fix through every downstream model. - **dbt_project.yml deprecation warning** (CustomKeyInConfigDeprecation) from a leftover auto-generated config block — fixed by restructuring the models: config to properly nest under staging:/marts: instead of a bare project-name key.

Extra dbt models added for dashboard support

- `stg_taxi_zones` — loads the official NYC TLC taxi zone lookup table (locationid → borough/zone), loaded into Snowflake via a separate `load_zones.py` script
- `agg_fare_by_borough` — joins `fct_trips` to `stg_taxi_zones` on `pu_location_id = locationid`, relabels 'Unknown' boroughs as 'Other'
- `agg_peak_hours` — originally summed trip counts by day-of-week/hour-of-day across the whole month (which inflated numbers, e.g. summing every Friday together); later corrected to divide by the actual number of occurrences of each weekday in the month, producing a true **average trips per weekday-hour** instead of a

monthly total. Also added a `day_order` column (Mon=1...Sun=7) for correct chronological sorting, and an `hour_label` column formatting hours as “12 AM”, “6 PM”, etc. instead of raw 0-23 integers.

Issue hit: loading the zone lookup CSV via a Python script initially failed with invalid identifier 'NAN' — pandas represented missing CSV values as NaN (a float), which the Snowflake connector couldn't bind as SQL. Fixed with `df = df.where(pd.notnull(df), None)` to convert missing values into proper Python None/SQL NULL.

5. Phase 4 — Visualize (Looker Studio)

Connected Looker Studio to Snowflake via the official connector, using custom SQL queries against each mart table as separate data sources.

Chart 1 — Daily trip volume (Time series): `agg_daily_summary` → `trip_date` (x-axis) vs `trip_count` (y-axis).

Chart 2 — Average fare by borough (Bar chart):
`agg_fare_by_borough` → `borough` (x-axis) vs `avg_fare` (y-axis). Added data labels, descriptive title, and a caption explaining EWR (Newark Airport) and the “Other” category.

Chart 3 — Peak hours heatmap (Pivot table with heatmap):
`agg_peak_hours` → rows = hour, columns = day of week, metric = trip count, colored with a high-contrast red/yellow/green scale (red = busiest). Row/column sorting required a workaround: displaying `DAY_OF_WEEK/HOUR_LABEL` as the visible dimension while sorting by the underlying numeric `DAY_ORDER/HOUR_OF_DAY` fields — this is because sorting a text field alphabetically doesn't match calendar order.

Issues hit and fixed: - Bad 2008 date (see above) initially wrecked the line chart's x-axis — fixed at the dbt layer. - Sort order for both day-of-week and hour-of-day defaulted to alphabetical/by-metric rather than calendar order — fixed by adding numeric helper columns (`day_order`, later also sorting hour by the underlying integer) and pointing the chart's sort settings at those, while keeping the readable text field as the display dimension. - Heatmap originally summed all occurrences of each weekday together (e.g. all 5 Fridays in May added up), producing misleadingly large numbers — fixed by dividing by the actual count of each weekday in the month, producing a true daily average. - x-axis on the line chart showed full dates (May 1, May 2...) which crowded and didn't have a plain-number format option in this version of Looker Studio — fixed by adding a `day_of_month` column via dbt (`day(trip_date)`) and using that as the chart's dimension instead.

6. Phase 5 — Document & Publish

- Created a public GitHub repo (nyc-taxi-pipeline)
- Wrote and iteratively expanded a full `README.md`: project overview, architecture, tools, project structure, data dictionary, data quality section, how-to-run instructions, dashboard link, visualizations with screenshots, key insights
- Added `.env.example` (placeholder credentials only, safe to commit) so anyone cloning the repo knows what environment variables to set
- Pushed to GitHub using a Personal Access Token (GitHub no longer accepts plain account passwords for git push)

Issues hit and fixed: - Initial push included the 66MB raw parquet file, triggering a GitHub file-size warning — fixed by `git rm --cached` on the file and adding `data/*.parquet` to `.gitignore`, keeping the raw data local-only (standard practice — raw data generally isn't committed to source control). - `dbt.log` had also been accidentally committed — untracked with `git rm -r --cached logs` and added `logs/` to `.gitignore`. - GitHub authentication: plain username/password no longer works for `git push`. Required generating a Personal Access Token (Settings → Developer settings → Personal access tokens → classic, with repo scope) and using that as the password when prompted.

7. Post-launch polish pass (making it resume-tier)

After the initial 5 phases were complete, a second pass was done to elevate the project quality:

1. **Fixed heatmap data** to show true averages instead of monthly sums (see Phase 3 above)
2. **Heatmap visual polish** — high-contrast red/yellow/green color scale, 12-hour AM/PM labels, descriptive title (“Average Trip Volume by Day of Week and Hour”), and a caption explaining the color direction
3. **Bar chart polish** — relabeled “Unknown” borough to “Other” at the dbt level, added a descriptive title, added on-bar data labels, added an explanatory caption
4. **Line chart polish** — added a `day_of_month` dbt column for cleaner x-axis labels, descriptive title (“Daily Taxi Trip Volume — May 2026”)
5. **Stronger dbt tests** — added 3 new tests beyond the original 7 basic `not_null/unique` checks:
 - `accepted_values` on `payment_type` (must be 1-6)
 - `relationships` test linking `fct_trips.pu_location_id` to `stg_taxi_zones.locationid`
 - Custom singular test `fare_amount_reasonable`, checking for negative fares or fares over \$500 paired with distances under 5 miles (refined after investigation — a flat fare cap alone incorrectly flagged legitimate long-distance trips)
6. **Fixed the `dbt_project.yml` deprecation warning** for a fully clean `dbt run/dbt test` output
7. **Published a real, shareable Looker Studio link** (“Anyone with the link can view”)
8. **Added an architecture diagram** (generated as an SVG, saved as a screenshot) replacing the plain-text pipeline description
9. **Added a data dictionary table** to the README, documenting key derived columns
10. **Added an MIT LICENSE file**
11. **Rewrote the README** with detailed per-chart explanations, a full data quality write-up, and cleaner formatting (nested list instead of an ASCII tree for the project structure, since GitHub renders ASCII trees poorly)

Key finding from the stronger tests: the custom fare test flagged 9 trips (out of ~3 million) with fares up to \$5,525.99 paired with trips lasting under a minute and covering under half a mile — clear metering/data-entry errors. Rather than silently filtering these out, the test was left intentionally failing and the finding documented in the README, mirroring how a real production data quality monitor surfaces (rather than hides) anomalies.

8. Final state

- **Rows processed:** 3,072,944 taxi trips (May 2026)
- **dbt models:** 6 (2 staging, 4 marts)
- **dbt tests:** 10 total — 9 passing, 1 intentionally failing (documented finding)
- **Dashboards:** 3 charts in Looker Studio, live and publicly shareable
- **Repo:** public on GitHub, fully documented, MIT licensed

Skills demonstrated across the project: Python data ingestion, SQL, dbt modeling (staging/marts pattern), data quality testing and investigation, cloud data warehousing (Snowflake), BI/dashboarding (Looker Studio), git/GitHub workflow, technical documentation, and real debugging across the stack (SSL certs, data type binding, bad source data, tool-specific sorting quirks).